Тема 14: ООП. Подтипов и параметричен полиморфизъм. Множествено наследяване.

ТЕОРИЯ

1. Статично vs Динамично свързване

	Статично свързване	**Динамично свързване**
Кога	По време на компилация	По време на изпълнение
Реализация	Обикновени (невиртуални) функции	Виртуални функции
Зависи от	Типа на **променливата**	Действителния **тип на обекта**

```
class Base {
public:
    void f() const { cout << "Base::f"; }      // статично
    virtual void g() const { cout << "Base::g"; } // динамично
};

class Der : public Base {
public:
    void f() const { cout << "Der::f"; }       // скрива Base::f
    void g() const override { cout << "Der::g"; } // override
};

void fn(const Base& ref) {
    ref.f();  // ВИНАГИ Base::f (статично — типът на ref е Base)
    ref.g();  // Зависи от реалния тип (динамично)
}

Der d;
fn(d);  // Извежда: Base::f   Der::g
```

---

2. Подтипов полиморфизъм

Полиморфизъм = 1 интерфейс, различни имплементации.

Изисквания:

1. Обща базова класа с `virtual` методи
2. Наследниците `override` методите
3. Достъп чрез **указател или референция** към базовия клас

```
class Animal {
public:
    virtual void speak() const { cout << "..."; }
    virtual ~Animal() {}   // ЗАДЪЛЖИТЕЛНО виртуален деструктор!
};
```

```
class Dog : public Animal {
public:
    void speak() const override { cout << "Woof!"; }
};

class Cat : public Animal {
public:
    void speak() const override { cout << "Meow!"; }
};

class Mouse : public Animal {
    // НЕ override — ще се използва Animal::speak()
};
```

Употреба с масив от указатели:

```
Animal* animals[] = { new Dog(), new Cat(), new Mouse() };
for (int i = 0; i < 3; i++) {
    animals[i]->speak();   // Dog: Woof! Cat: Meow! Mouse: ...
}
for (int i = 0; i < 3; i++) {
    delete animals[i];     // ~Dog, ~Cat, ~Mouse + ~Animal (виртуален!)
}
```

> ⚠ **Виртуален деструктор в Base е задължителен**, ако ще изтриваме наследници чрез Base*!
> Без него `delete base_ptr` вика само `~Base` → **undefined behavior** / **memory leak**.

---

3. `override` и `final`

```
class Base {
    virtual void f() {}
};
class Der : public Base {
    void f() override {}   // OK — f е виртуална в Base
    // void g() override {} — ГРЕШКА при компилация: g не е виртуална в Base
};
class Sealed final : public Base {  // от Sealed не може да се наследява
    void f() override final {}      // f не може да се override в наследници
};
```

---

4. Абстрактни методи и класове

Чисто виртуален метод (Pure Virtual):

```
virtual void f() = 0;   // абстрактен метод — задължителен за override
```

Абстрактен клас = клас с поне един чисто виртуален метод.

- **НЕ може** да се създават обекти от него: `Abstract a;` → грешка!
- Може да има указатели/референции: `Abstract* ptr`

```
class Shape {
```

```
public:
    virtual double area() const = 0;     // чисто виртуален
    virtual void draw() const = 0;
    virtual ~Shape() {}
};

class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
    void draw() const override { cout << "O"; }
};

// Shape s;         // ГРЕШКА — абстрактен клас
Shape* s = new Circle(5.0);  // OK
s->area();   // динамично → Circle::area()
```

Ако наследник **не имплементира** всички чисто виртуални методи → и той е абстрактен.

---

5. Масиви от обекти vs указатели

Масив от обекти — ХОМОГЕНЕН

```
Animal arr[3];  // само Animal обекти; НЕ може Dog, Cat...
```

Масив от указатели — ХЕТЕРОГЕНЕН (полиморфизъм!)

```
Animal* arr[] = { new Dog(), new Cat(), new Mouse() };
// Можем да миксираме типове от йерархията
```

Хетерогенен контейнер с clone (Prototype Pattern):

```
class Base {
public:
    virtual Base* clone() const = 0;  // за дълбоко копиране
    virtual ~Base() {}
};
class Dog : public Base {
public:
    Dog* clone() const override { return new Dog(*this); }
};

// Копиране на контейнер:
Base** copy = new Base*[size];
for (int i = 0; i < size; i++)
    copy[i] = original[i]->clone();
```

---

6. Параметричен полиморфизъм — Шаблони (Templates)

Параметричен полиморфизъм = **една имплементация за различни типове** (генеричен код).
Компилаторът генерира отделна версия за всеки конкретен тип.

Шаблонна функция (Function Template)

```
template <typename T>
const T& maxOf(const T& a, const T& b) {
    return a < b ? b : a;
}

maxOf(3, 5);          // T = int (извежда се автоматично)
maxOf(3.14, 2.71);    // T = double
maxOf<string>("ab", "cd"); // T = string (явно указан)
```

Компилаторът генерира отделна функция за int, double, string...
Типът T трябва да поддържа **<** оператора (или каквото е използвано в тялото).

Шаблонен клас (Class Template)

```
template <typename T>
class Stack {
    T*     data;
    int    top;
    int    capacity;
public:
    Stack(int cap = 16) : top(0), capacity(cap) {
        data = new T[cap];
    }
    ~Stack() { delete[] data; }

    void push(const T& val) { data[top++] = val; }
    T    pop()              { return data[--top]; }
    bool empty() const      { return top == 0; }
};

Stack<int>    si;
Stack<string> ss;
si.push(42);
ss.push("hello");
```

Шаблонна специализация

```
template <typename T>
void print(const T& val) {
    cout << "Generic: " << val;
}

template <>              // специализация за bool
void print<bool>(const bool& val) {
    cout << (val ? "true" : "false");
}

print(3.14);   // Generic: 3.14
print(true);   // true
```

---

7. Множествено наследяване

```
class Flyable { public: virtual void fly() {} };
class Swimmable { public: virtual void swim() {} };

class Duck : public Flyable, public Swimmable {
    void fly()  override { cout << "Duck flies"; }
```

```
    void swim() override { cout << "Duck swims"; }
};
```

Двусмислица при едноименни членове

```
class A { public: void f() {} };
class B { public: void f() {} };
class C : public A, public B {};

C c;
c.f();         // ГРЕШКА — неясно кое f
c.A::f();      // ОК — явно указваме A::f
c.B::f();      // ОК — явно указваме B::f
```

Диамантен проблем

```
class Animal { protected: int age; };
class Dog    : public Animal {};
class Cat    : public Animal {};
class DogCat : public Dog, public Cat {};
// DogCat има ДВЕ копия на Animal — неясно кое age!
```

Решение: Виртуално наследяване

```
class Animal { protected: int age; };
class Dog : virtual public Animal {};  // virtual!
class Cat : virtual public Animal {};  // virtual!
class DogCat : public Dog, public Cat {};
// DogCat има ЕДНО копие на Animal
// DogCat е отговорен за конструктора на Animal
```

КАК СЕ РЕШАВАТ ЗАДАЧИ

> Task 2 на практическия изпит (2021–2024) — ООР C++. Честите задачи: trace на виртуални функции, имплементация с шаблони, наследяване с полиморфизъм.

Тип 1: Trace на виртуални функции

Ключово правило:

- Невиртуална функция → **типът на променливата** (статично)
- Виртуална функция → **реалният тип на обекта** (динамично)

```
class A {
public:
    void f() { cout << "A::f "; }
    virtual void g() { cout << "A::g "; }
};
class B : public A {
public:
    void f() { cout << "B::f "; }
    void g() override { cout << "B::g "; }
};
class C : public B {
public:
```

```
    void g() override { cout << "C::g "; }
};

int main() {
    A* ptr = new C();
    ptr->f();   // A::f  (невиртуална — типът на ptr = A*)
    ptr->g();   // C::g  (виртуална — реален тип = C)

    B& ref = *new C();
    ref.f();    // B::f  (невиртуална — типът на ref = B&)
    ref.g();    // C::g  (виртуална — реален тип = C)
}
```

Стъпки за trace:

1. Виртуална ли е функцията? → Погледни базовата класа
2. Да → следвай реалния тип на обекта по йерархията надолу
3. Не → следвай типа на указателя/референцията

Тип 2: Имплементация на шаблонен клас с Rule of Three

```
template <typename T>
class DynArray {
    T*   data;
    int  size;
    int  capacity;

public:
    DynArray() : data(nullptr), size(0), capacity(0) {}

    DynArray(const DynArray& other) : size(other.size), capacity(other.capacity)
{
        data = new T[capacity];
        for (int i = 0; i < size; i++) data[i] = other.data[i];
    }

    DynArray& operator=(const DynArray& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            capacity = other.capacity;
            data = new T[capacity];
            for (int i = 0; i < size; i++) data[i] = other.data[i];
        }
        return *this;
    }

    ~DynArray() { delete[] data; }

    void push_back(const T& val) {
        if (size == capacity) resize();
        data[size++] = val;
    }

    T& operator[](int i) { return data[i]; }

private:
    void resize() {
        capacity = capacity == 0 ? 1 : capacity * 2;
```

```
        T* newData = new T[capacity];
        for (int i = 0; i < size; i++) newData[i] = data[i];
        delete[] data;
        data = newData;
    }
};
```

Тип 3: Полиморфна йерархия с чисто виртуален метод

```
class Employee {
    string name;
public:
    Employee(const string& n) : name(n) {}
    virtual double salary() const = 0;    // абстрактен!
    virtual ~Employee() {}
    string getName() const { return name; }
};

class FullTime : public Employee {
    double monthly;
public:
    FullTime(const string& n, double m) : Employee(n), monthly(m) {}
    double salary() const override { return monthly; }
};

class PartTime : public Employee {
    double hourly;
    int    hours;
public:
    PartTime(const string& n, double h, int hrs)
        : Employee(n), hourly(h), hours(hrs) {}
    double salary() const override { return hourly * hours; }
};

// Полиморфна употреба:
Employee* staff[] = {
    new FullTime("Alice", 3000),
    new PartTime("Bob", 20, 80)
};
for (int i = 0; i < 2; i++) {
    cout << staff[i]->getName() << ": " << staff[i]->salary() << endl;
    delete staff[i];
}
```

Бърза справка

Концепция	**Ключов факт**
`virtual void f()`	Динамично свързване
Невиртуална функция	Статично — следва типа на ptr/ref
`= 0`	Чисто виртуален — клас е абстрактен
Абстрактен клас	Не може да се инстанцира
Виртуален деструктор	**Задължителен** при полиморфна йерархия
`override`	Компилаторна проверка; предотвратява

Концепция	**Ключов факт**
	грешки
`clone()`	Prototype pattern за копиране на хетерогенен контейнер
`template <typename T>`	Параметричен полиморфизъм
Шаблонна специализация	Различно поведение за конкретен тип
Множествено наследяване	`class D : public B, public C`
Диамантен проблем	Две копия на базовия клас
`virtual public`	Решение — едно копие на базовия клас